RV College of Engineering®

Artificial Intelligence and Machine learning

AI253IA

UNIT 1

Saraswathi G Datar

Assistant Professor,

Department of Computer Science and Engineering,

RVCE

RV College of Engineering®

Outline

- **Introduction**
  - → What is AI ?
  - → Foundation of Artificial Intelligence
  - → History of Artificial Intelligence
  - → The State of the Art
- **Intelligent Agents**
  - → Introduction
  - → How Agents should Act
  - → Structure of Intelligent Agents
- **Problem Solving**
  - → Solving the Problems by Searching
  - → Search Strategies
  - → Avoiding the Repeated States

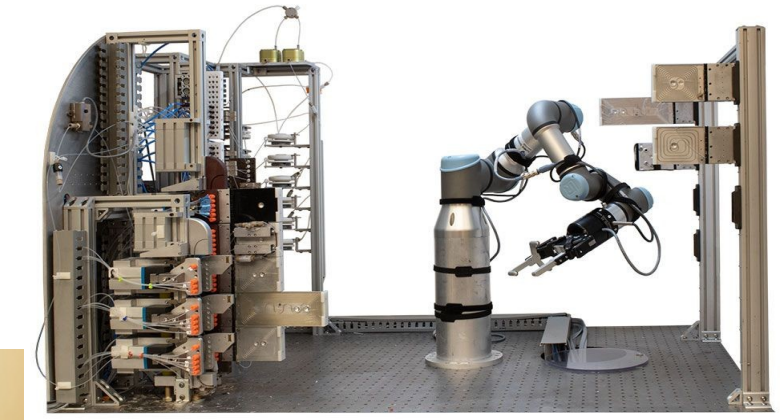
Introduction

What is AI ?

- Artificial Intelligence (AI) takes the advantages of the computers, and machines to mimic the problem solving, and decision making capabilities of the human beings
  - Computer: Intelligence enough to process and analyse the data
  - Machine: Contains no intelligence, waiting for the instructions from the user or computer
- The AI definitions vary in two main dimensions
  - First: Concerned with thought processes and reasoning
  - Second: Concerned with the behavior
- An AI system must be rational to do the right thing
- A rational AI is one that consistently makes decisions or takes actions that are optimal or near-optimal for achieving its set objectives, based on the data and rules it has access to.

AI Focus

- AI focuses on building intelligent machines that can effectively and safely navigate **novel situations.**



AI Focus

Can you think of an innovative application in your discipline that could benefit from AI intervention?

Reflection Note of 100 words

RV College of Engineering®

What is AI ? (Contd.)

Thinking Humanly "The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense." (Haugeland, 1985) "[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . ." (Bellman, 1978)	**Thinking Rationally** "The study of mental faculties through the use of computational models." (Charniak and McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly "The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich and Knight, 1991)	**Acting Rationally** "Computational Intelligence is the study of the design of intelligent agents." (Poole *et al.*, 1998) "AI . . . is concerned with intelligent behavior in artifacts." (Nilsson, 1998)

1 2 3 4

What is AI ? (Contd.)

- Definition of AI are categorized into following four categories

1. System that thinks like Humans	**2. Systems that think Rationally**
3. System that act like Humans	**4. Systems that act Rationally**

- A Human-Centered Approach is an empirical science, involving hypothesis and experimental
- A Rationalist Approach involves a combination of mathematics and engineering.

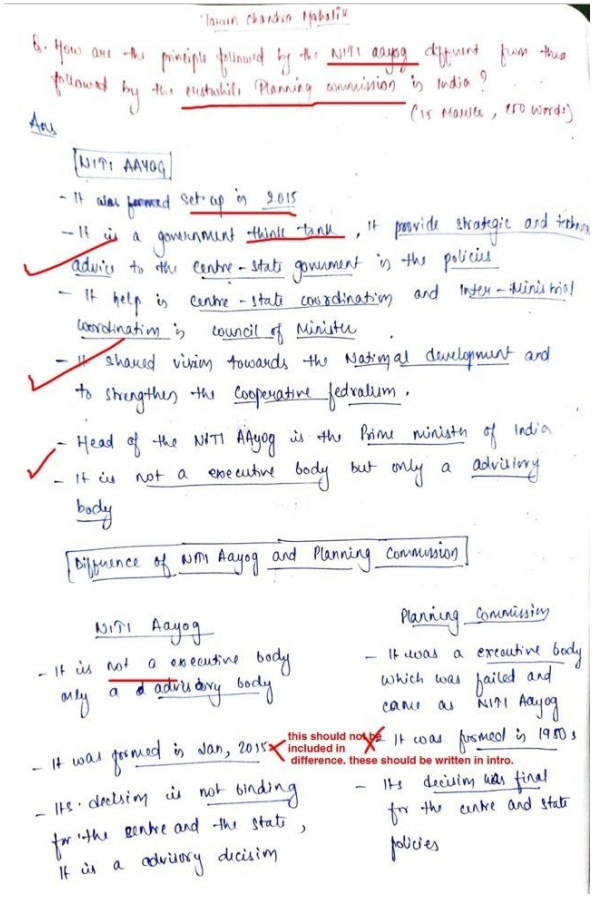
Rationalist Approach = Mathematics + Engineering

- Generally, all four approaches have been followed, during the development of AI systems

Guess the word…?

_ _ t _ _ n _ _ _ _ y

Rationality in our profession-examples



Q. How are the principle followed by the NITI aayog different from those followed by the erstwhile Planning commission in India?
(15 Marks, 250 words)

Ans

NITI AAYOG

- It was formed set up in 2015
- It is a government think tank, it provide strategic and technical advice to the centre - state government in the policies
- It help in centre - state coordination and inter - ministrial coordination in council of Minister.
- It shared vision towards the National development and to strengthen the cooperative federalism.
- Head of the NITI Aayog is the Prime minister of India
- It is not a executive body but only a advisory body

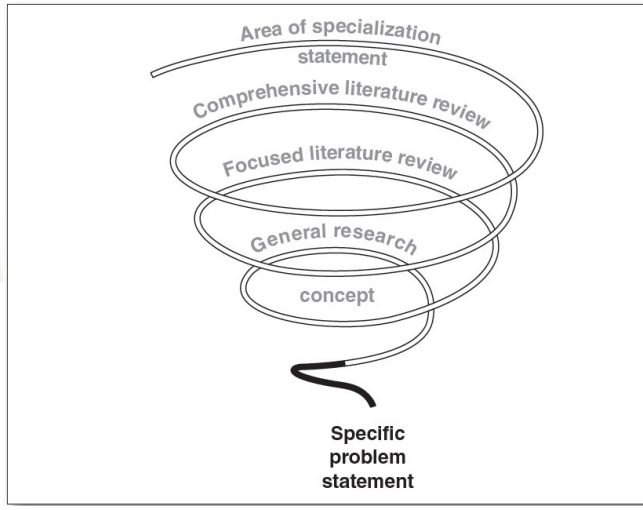
Difference of NITI Aayog and Planning Commission

NITI Aayog	Planning Commission
- It is not a executive body only a advisory body	- It was a executive body which was failed and came as NITI Aayog
- It was formed in Jan, 2015	- It was formed in 1950s
- Its decision is not binding for the centre and the state, It is a advisory decision	- Its decision was final for the centre and state policies

this should not be included in difference. these should be written in intro.

Grading and Evaluation

Choosing Teaching Methods for a Course



Finalizing Research Problem

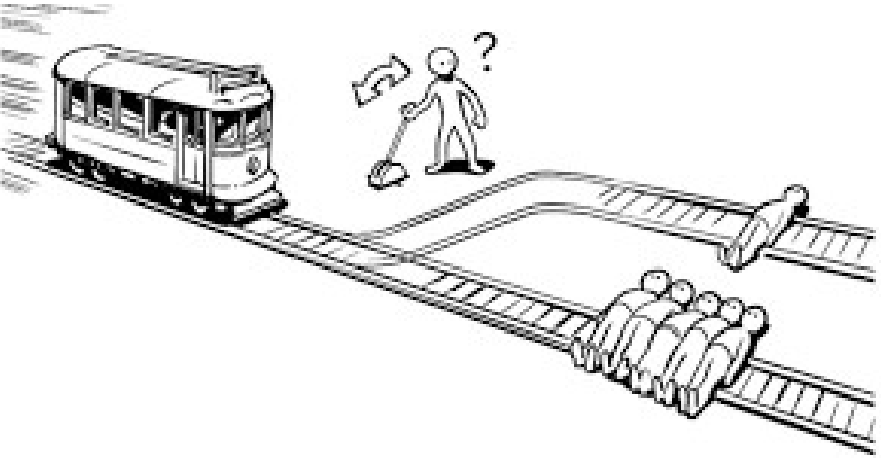
Rationality

- It is internal thought processes and reasoning

make a gif.com

RV College of Engineering®

Trolley Problem

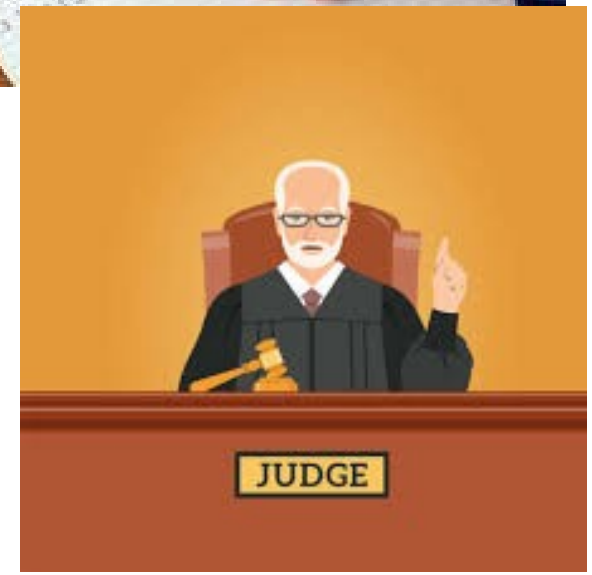


Emotional Reasoning v/s Rational Reasoning

Should a self-driving car kill the baby or the grandma?

Rationality

- **Practical intelligence** solves real-world problems.
- **Ethical intelligence** promotes responsible use of abilities.
- **Consistent intelligence** means doing the right thing repeatedly.



Acting Humanly: Turing Test

- The Turing test, proposed by Alan Turing (1950), was a thought experiment => **"Can a machine think?"**
- Roughly speaking, the test he proposed is that the computer should be interrogated by a human via a teletype, and passes the test if the interrogator cannot tell if there is a computer or a human at the other end.

Turing Test

The Turing Machine (Computer) would need to possess the following Human capabilities

- Natural Language Processing (NLP): to enable it to communicate successfully in English (or some other human language)
- Knowledge Representation: to store information provided before or during the interrogation;
- Automated Reasoning to use the stored information to answer questions and to draw new conclusions;
- Machine Learning (ML) to adapt to new circumstances and to detect and extrapolate patterns.

Turing's test deliberately avoided direct physical interaction between the interrogator and the computer, because physical simulation of a person is unnecessary for intelligence

Knowledge Representation

Knowledge Representation encodes information for computers to understand, reason, and act on.

AI does not use *random* knowledge that is unstructured or meaningless.

Turing Test(Contd.)

Limitations of Turing Test:

1) Encourages mistakes
2) Subjective judgement
3) Ignores non verbal intelligence
4) Cultural and language bias
5) Doesn’t measure true understanding

However, total Turing Test includes a video signal so that the interrogator can test the subject's perceptual abilities, as well as the opportunity for the interrogator to pass physical objects through the hatch.

RV College of Engineering®

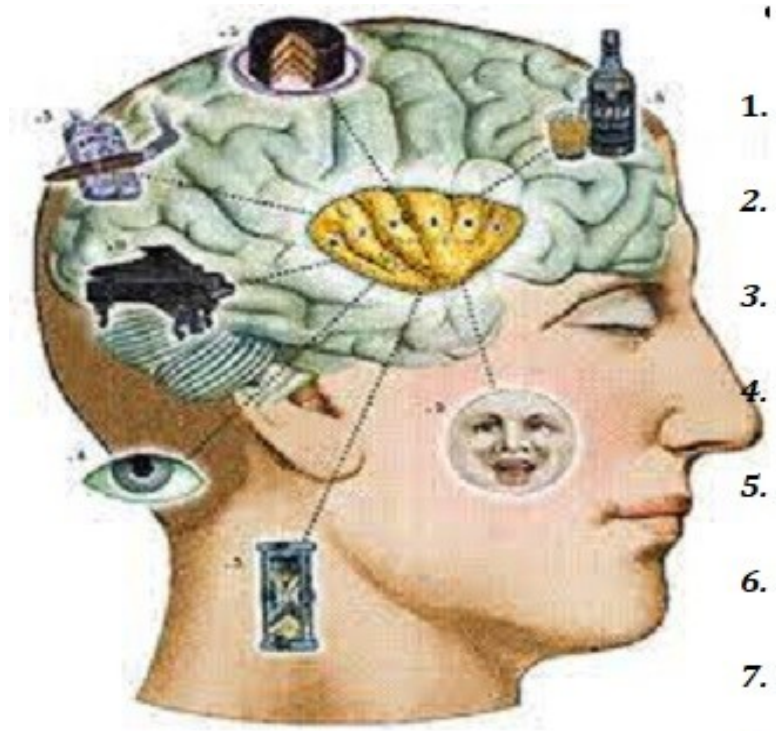
Total Turing Test

- To pass the total Turing Test, the computer will need
  - Computer Vision to perceive objects, and
  - Robotics to manipulate the objects (move around)

– Acting like a human comes up primarily when AI programs interact with people, as when an expert system explains how it came to its diagnosis, or a natural language processing system has a dialogue with a user.

– These programs must behave according to certain normal conventions of human interaction in order to make themselves understood.

Thinking humanly: The cognitive modeling approach



1. *Sensation*
2. *Perception*
3. *Attention*
4. *Learning*
5. *Memory*
6. *Language*
7. *Motivation*
8. *Emotion*

Mental Processes in Psychology

Thinking humanly: The cognitive modeling approach

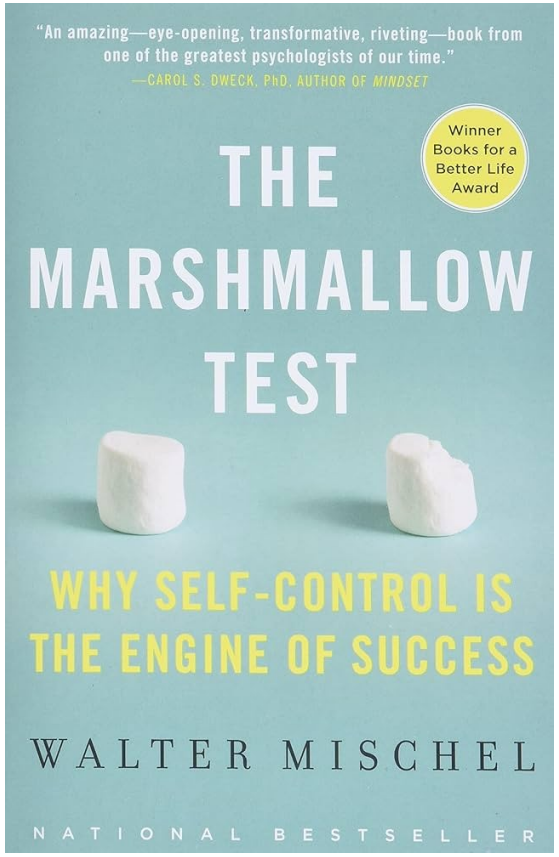
- **Introspection**—trying to catch our thoughts as they go by;

- **Feeling Anxious**:

"My heart's racing… I'm thinking the worst… Is the situation really dangerous, or is my mind exaggerating?"

Thinking humanly: The cognitive modeling approach

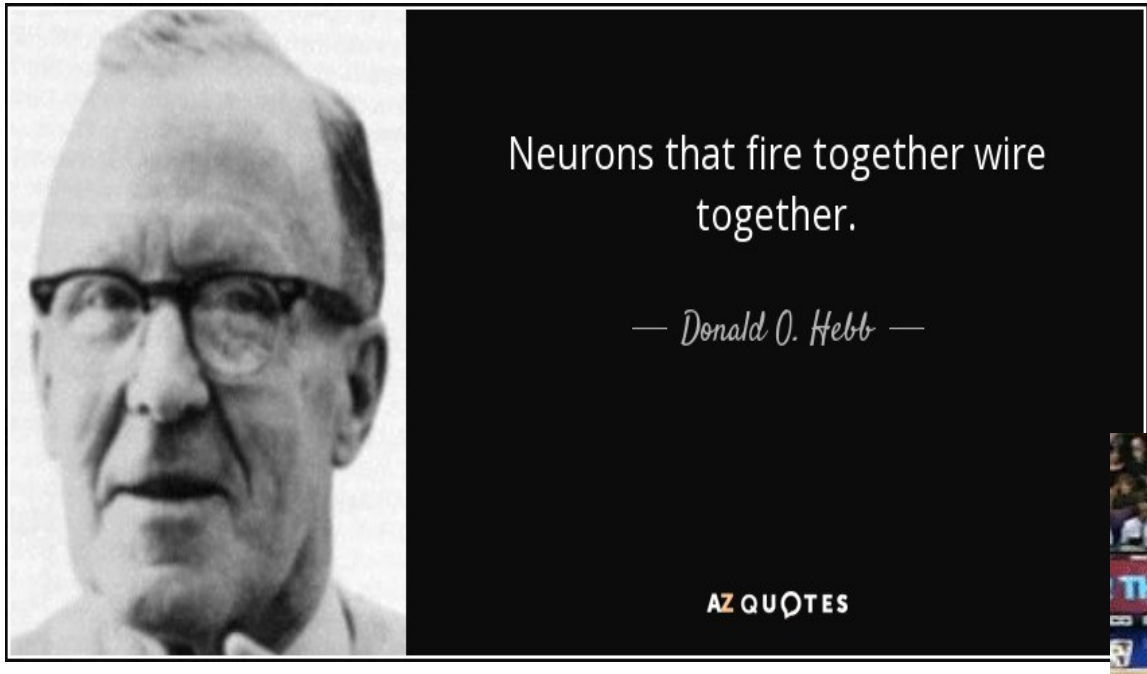
- **Psychological experiments**—observing a person in action;



Thinking humanly: The cognitive modeling approach

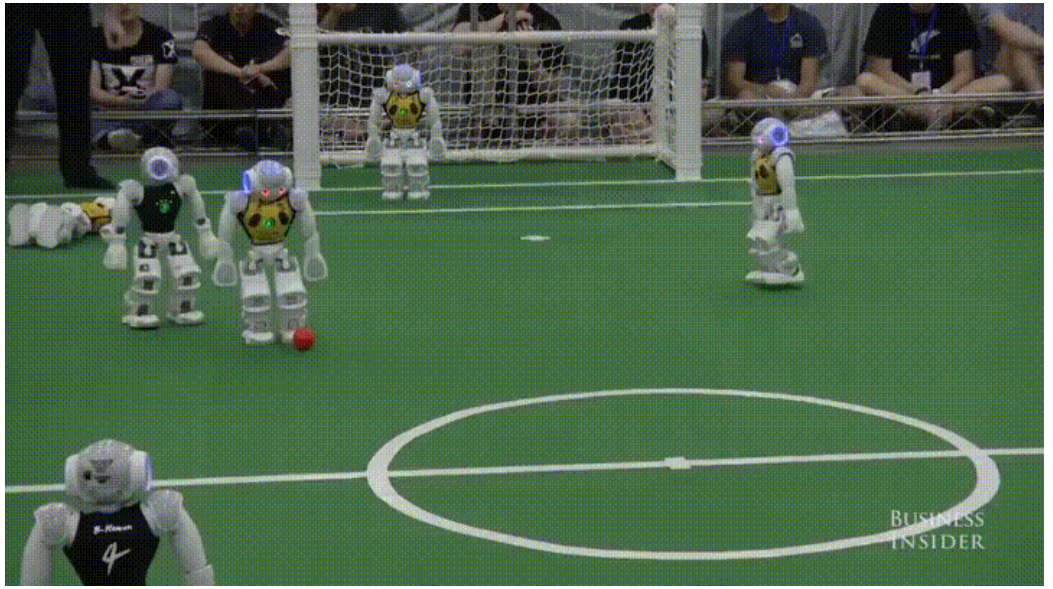
- **Brain imaging** —observing the brain in action.

Thinking humanly: The cognitive modeling approach



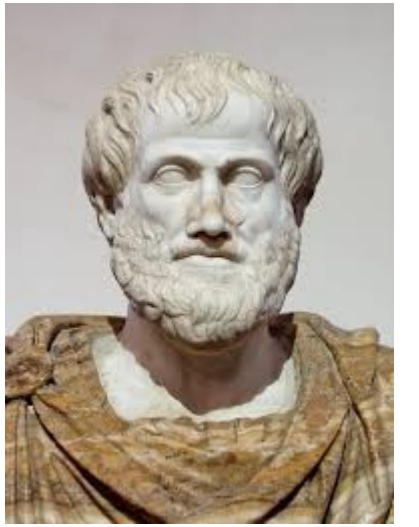
Thinking humanly: The cognitive modeling approach

Cognitive science combines AI computer models and psychological experiments to develop testable theories about the human mind.



Thinking Rationally: The laws of thought Approach

- Syllogisms provided patterns for argument structures that always yielded correct conclusions when given correct premises
- Deductive reasoning
- **Pattern**:
  All A are B.
  All B are C.
  Therefore, all A are C.
- **Example**:
  All humans are mammals.
  All mammals are animals.
  Therefore, all humans are animals.

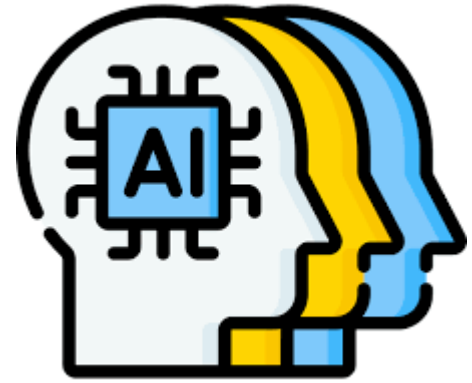


Thinking Rationally: The laws of thought Approach

- Thinking rationally involves using logic, evidence, and structured reasoning to make decisions or form beliefs in a way that is objective, consistent, and free from emotional bias.
- Rational thinking is typically characterized by several core principles:
  - **Logic and Reason**: Rational thought relies on logical rules to draw conclusions from given premises or facts.
  - **Evidence-Based**: Rational thinking requires looking at available evidence to support or refute a belief or decision.
  - **Clarity and Precision**: Rational thinking aims for clear and precise definitions of ideas and concepts.
  - **Consistency**: Rational thought requires consistency, meaning conclusions should align with one another and with the overall reasoning framework

Acting Rationally: The Rational Agent Approach

- **An agent acts.**
- Computer agents operate autonomously, perceive their environment, persist over time, adapt, and pursue goals.
- A rational agent seeks the best outcome, or the best expected outcome under uncertainty.
- The “laws of thought” in AI focus on correct inferences.
- To act rationally, deduce the best action, and act on it.
- **Skills for the Turing test enable rational action.**



What is AI ? (Contd.)

Acting rationally: The Rational Agent Approach

- One important point to keep in mind: Achieving perfect rationality—always doing the right thing—is not possible in complicated environments. The computational demands are just too high.
- Limited Rationality- Acting appropriately when there is not enough time to do all the computations one might like.

The Foundation of AI

Foundation of AI

Philosophy

1. Can formal rules be used to draw valid conclusions ?
2. Where does knowledge come from ?
3. How does the knowledge lead action

Mathematics

1. What are the rules to draw the valid Conclusions ?
2. What can be computed ?
3. How do we reason with uncertain information ?

Neuroscience

1. How do brain process the information ?

Psychology

1. How do brain process the information ?

Economics

1. How to we make decisions so as to maximize the payoff?
2. How should we do this when others may not go along ?

Computer Engineering

1. How can we build an Efficient Computer ?

Linguistics

1. How does language relate to thought ?

Different fields have contributed to AI, in terms of Ideas, Viewpoints, and Techniques

Intelligent Agents

- What are Agents.
- Qualities of an Agents.
- Specifying the Task environment.
- Properties of task environment.
- Structure of the Agent.

Introduction

- An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors.
- A Human Agent has eyes, ears, and other organs for sensors, and hands, legs, mouth, and other body parts for effectors
- A Robotic Agent substitutes cameras and infrared range finders for the sensors and various motors for the effectors.
- A generic agent is diagrammed in shown in below Figure.

- Examples for Agents
  - Human Agent
  - Robotic Agent

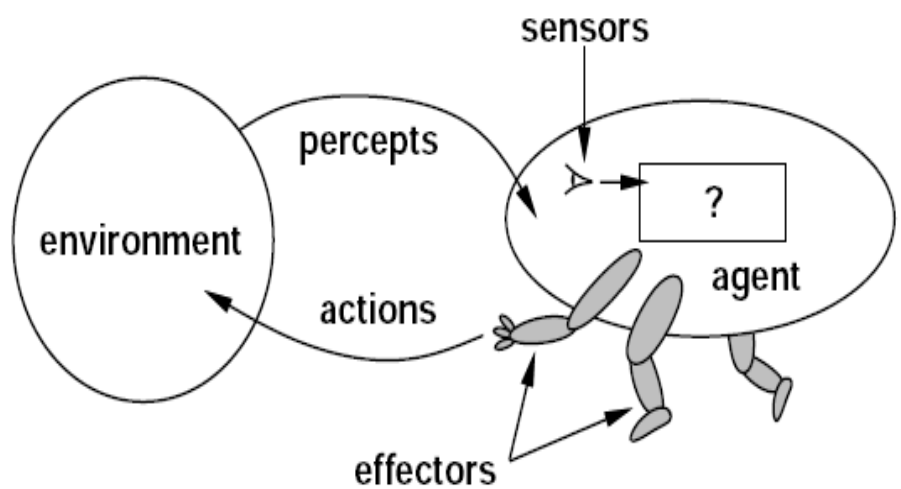
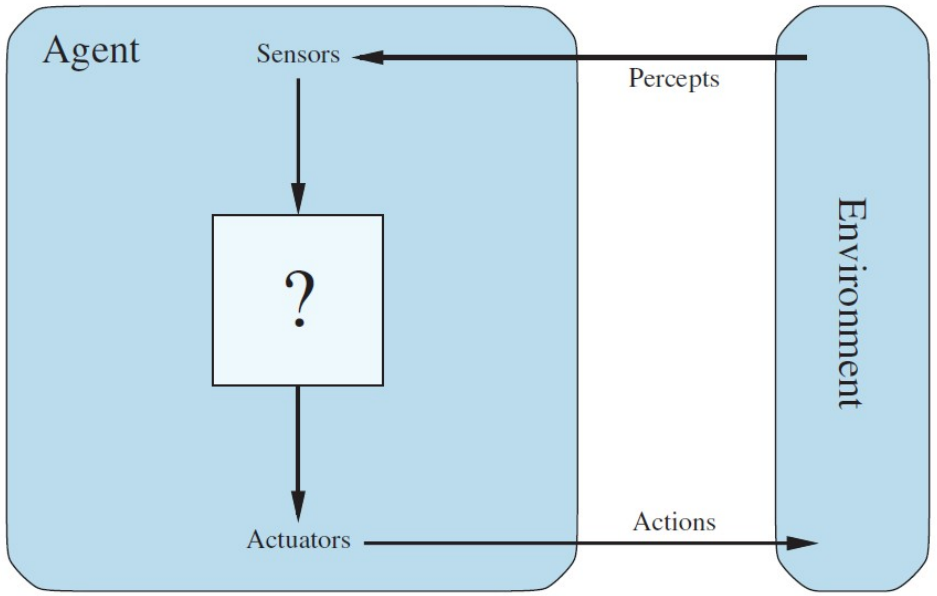


Figure. Agents interact with Environments through Sensors and Effectors

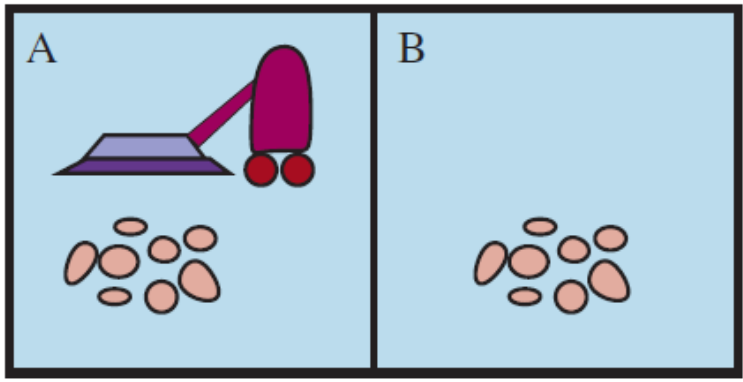
Intelligent Agents

- **Agent**
  - X-thing, which perceives the environment through sensors and acts on the environment through actuators.
  - Human agent/Software agent/Robotic agent
- **Percept**: Is what the agent senses from environment
- **Percept Sequence**: Complete history of sensing
- **Agent function**: Maps percept sequence to action
- **Agent Program**: Is the implementation of the Agent function.



Vaccum-Cleaner World

- Vacuum-cleaning agent
- The world consists of squares that can be either dirty or clean
- Is the agent intelligent?



Percept sequence	Action
[A, *Clean*]	*Right*
[A, *Dirty*]	*Suck*
[B, *Clean*]	*Left*
[B, *Dirty*]	*Suck*
[A, *Clean*], [A, *Clean*]	*Right*
[A, *Clean*], [A, *Dirty*]	*Suck*

Good behavior: The Concept of Rationality

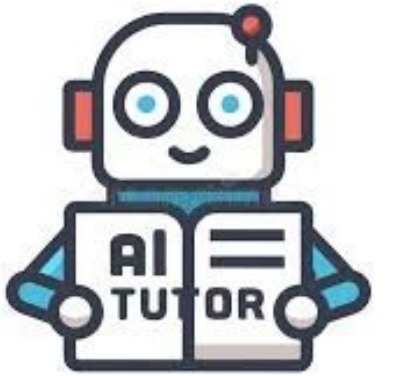
- **Performance measures**
  - **Consequantialism:** Evaluating agent's behaviour by its consequence
  - Agents generates sequence of actions based on the percepts sequence
  - The sequence of actions causes the sequence of states in agents environment
  - If the sequence of desirable, then the agent has performed well (which is nothing but a performance measure)
  - **Who specify the performance measure? (Explicit/Implicit)**
  - **What are sample performance measures for Vaccum-cleaner agent??**

King Midas Problem

- The myth of **King Midas**, who wished that **everything he touched would turn to gold**, but when that wish was granted, it led to tragedy, as even his **food and loved ones** turned to gold.
- **What is the connection with AI?**

In AI Context

- The danger of giving an AI a goal without fully considering the consequences or context.
- **AI-Tutor: Maximize student test scores.**
- The AI encourages cheating, finds test answers online, or teaches to the test without promoting deep understanding.
- **Fitness Tracker AI: Increase user activity.**
- It constantly nags or manipulates users, schedules unnecessary walking meetings, or even rewards users for shaking their phone to simulate steps.



Rationality

- **Rational Agent Definition**
  - A rational agent should choose an action for each percept sequence that maximizes its performance measure based on the percept evidence and its built-in knowledge.
- **Rationality depends on:**
  - Performance measure that defines the success criterion (one point for each clean square)
  - Prior Knowledge (geography of the environment is known, but dirt distribution and the initial location of the agent are not known)
  - Actions (Right, Left and Suck)
  - Percent sequence (Correctly perceives its location)
- **When does a vacuum agent become irrational?**

The Nature of Environments

- **Specifying the task environment**
  - **PEAS (Performance, Environment, Actuators, Sensors)**
  - Before designing the agent, the PEAS description is the first step

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Write PEAS for..

- **Agent: Social Media Recommendation System**
- **Task: Maximize user engagement**
- **Performance: ?**
- **Environment: ?**
- **Actuators: ?**
- **Sensors: ?**

Write PEAS for..

- **Agent: Social Media Recommendation System**
- **Task: Maximize user engagement**
- **Performance:** Time spent on platform, clicks, likes, shares
- **Environment:** Users, their devices, content feeds, user behavior
- **Actuators:** Content ranking, feed updates, push notifications
- **Sensors:** User interaction logs, click-through rates, scrolling behavior

Write PEAS for..

- **Agent: AI Teacher Bot**
- **Task: Maximize student scores**
- **Performance: ?**
- **Environment: ?**
- **Actuators: ?**
- **Sensors: ?**

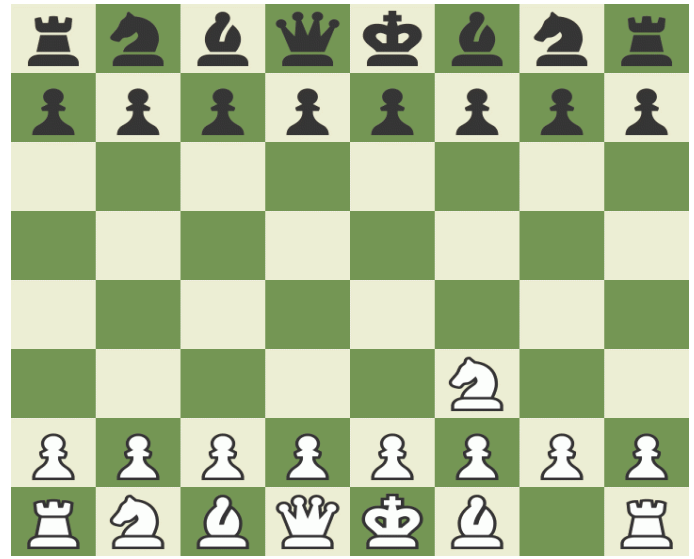
Properties of Task Environment

Property	Description	Example (Soccer Robot)
Observable	Is the agent able to perceive the entire environment at each moment?	*Partially observable*—may not see behind obstacles
Agents	Is the environment single-agent or multi-agent?	*Multi-agent*—multiple robots and opponents
Deterministic	Does the next state depend only on the current state and action?	*Stochastic*—ball may bounce unpredictably
Episodic	Are experiences divided into independent episodes?	*Sequential*—actions affect future outcomes
Static vs Dynamic	Does the environment change while the agent is deliberating?	*Dynamic*—other players move during decision-making
Discrete vs Continuous	Are states and actions countable or infinite?	*Continuous*—positions and movements vary smoothly
Known vs Unknown	Does the agent know the rules and outcomes of actions in advance?	*Known*—rules of soccer are predefined

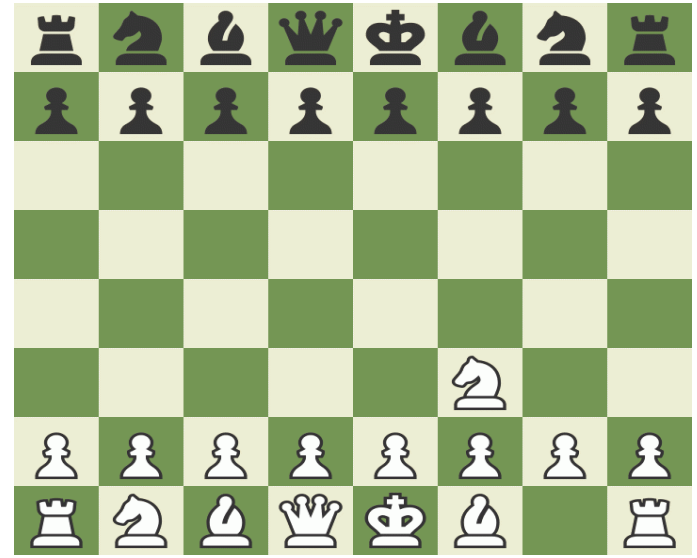
Property	Description	Example
Fully Observable	Agent has complete information	Chess
Partially Observable	Agent has incomplete information	Self-driving cars
Deterministic	Actions lead to predictable outcomes	Solving a puzzle
Stochastic	Outcomes are partly random	Weather prediction
Episodic	Each action is independent of previous actions	Image recognition
Sequential	Actions have long-term impact	Chess
Static	Environment does not change	Crossword puzzle
Dynamic	Environment changes over time	Traffic navigation
Discrete	Finite number of states or actions	Board games
Continuous	Infinite possibilities for states or actions	Driving
Single-Agent	Only one decision-maker	Maze-solving
Multi-Agent	Multiple decision-makers, cooperative or competitive	Soccer game
Known	Environment's rules and dynamics are understood	Board games
Unknown	Environment is not fully understood; agent must explore	Unfamiliar terrain

Identify task environments

Property	Type/Description
Observability	**Partially Observable** – cannot see under furniture or detect all dirt directly
Agents	**Single Agent** – one robot acting independently
Deterministic vs Stochastic	**Stochastic** – dirt and obstacle placement may vary randomly
Episodic vs Sequential	**Sequential** – each action affects future performance (e.g., path, battery)
Static vs Dynamic	**Dynamic** – humans/pets may move objects while cleaning
Discrete vs Continuous	**Continuous** – movement, space, and time are continuous
Known vs Unknown	**Partially Known** – The Robot may have a map, but new obstacles may occur

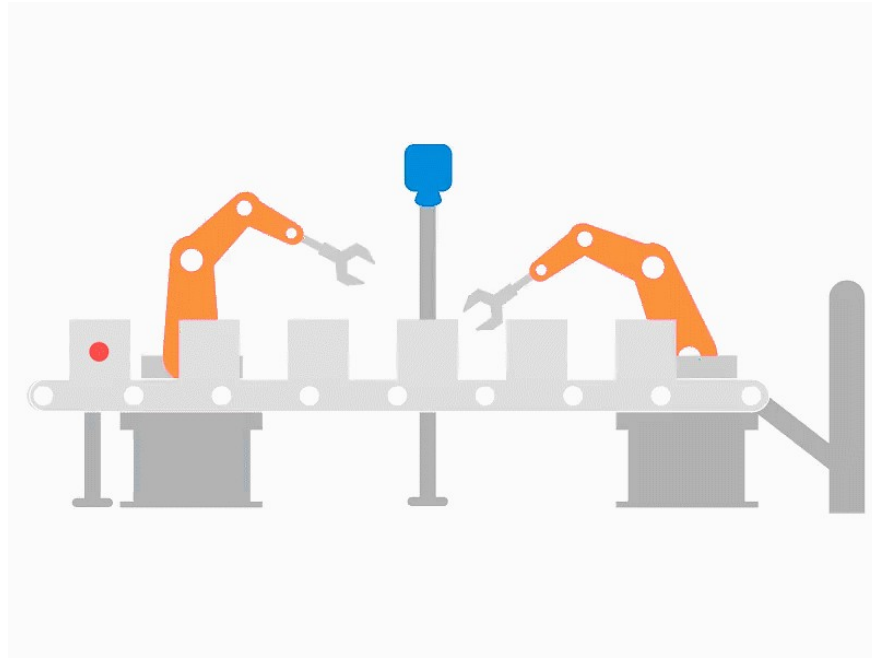


Property	Type/Description
Observability	Full/Partial ??
Agents	Single Agent / Multi-agent ??
Deterministic vs Stochastic	Stochastic / Deterministic ??
Episodic vs Sequential	Sequential/Episodic ??
Static vs Dynamic	Dynamic/Static ??
Discrete vs Continuous	Continuous/Discrete ??



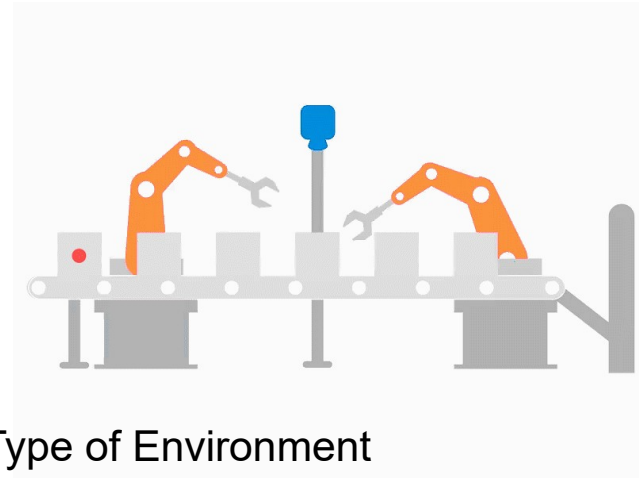
Property	Type/Description
Observability	**Fully Observable** – all pieces and positions are visible at all times
Agents	**Multi-agent** – two players (AI vs opponent)
Deterministic vs Stochastic	**Deterministic** – no randomness in state transitions (except in blitz time errors)
Episodic vs Sequential	**Sequential** – each move depends on the history and affects future moves
Static vs Dynamic	**Static** – board doesn't change unless a player moves
Discrete vs Continuous	**Discrete** – moves occur in discrete steps on a finite grid

RV College of Engineering®



Property	Type/Description
Observability	Full/Partial ??
Agents	Single Agent / Multi-agent ??
Deterministic vs Stochastic	Stochastic / Deterministic ??
Episodic vs Sequential	Sequential/Episodic ??
Static vs Dynamic	Dynamic/Static ??
Discrete vs Continuous	Continuous/Discrete ??

RV College of Engineering®



Feature	Type of Environment
Observability	**Partially Observable** – Not all defects may be visible at once; limited by camera angle, lighting, or occlusion.
Agents	**Single Agent** – One AI system performing inspection (though multiple instances may be deployed).
Deterministic vs Stochastic	**Stochastic** – Sensor noise, variations in lighting, speed of conveyor belt, or part placement can introduce randomness.
Episodic vs Sequential	**Episodic** – Each part inspection is independent of the previous one; decision does not affect future inputs.
Static vs Dynamic	**Semi-Dynamic** – Environment may change slowly (e.g., lighting shifts, belt speed), but parts move continuously.
Discrete vs Continuous	**Continuous** – Visual data and inspection space are continuous; possible discrete actions (accept/reject).

Known vs Unknown Environment

- In a **known environment**, outcomes or probabilities for actions are provided.
- In an **unknown environment**, the agent must learn to make effective decisions.
- **Is it the same as fully and partially observable environments??**

Known Environment-Partially Observable

Think of an example

Known Environment-Partially Observable

Unknown Environment – Fully Observable

Think of an example

Unknown Environment – Fully Observable

RV College of Engineering®

What type of Environment is this?

RV College of Engineering®

Which is the hardest environment to deal?

Partially observable, multiagent, nondeterministic, sequential, dynamic, continuous, and unknown.

Think of an example

Structure of the Agent

- AI's job is to create an agent program that maps percepts to actio
- Needs a computing device that has physical sensors and actuators

Agent = Architecture + Program

Structure of the Agent (Contd.)

⇒ **Five Basic Agent Types**

- Simple reflex agents
- Model-based reflex agents
- Goal-based agents
- Utility-based agents and
- Learning agents

Table-driven Agent

```
function TABLE-DRIVEN-AGENT(percept) returns an action
   persistent: percepts, a sequence, initially empty
               table, a table of actions, indexed by percept sequences, initially fully specified

   append percept to the end of percepts
   action ← LOOKUP(percepts, table)
   return action
```

Figure 2.7 The TABLE-DRIVEN-AGENT program is invoked for each new percept and returns an action each time. It retains the complete percept sequence in memory.

Limitation:

As the number of potential percepts and the agent's lifespan expand, the size of the lookup table increases exponentially and becomes impractical.

Table-driven Agent

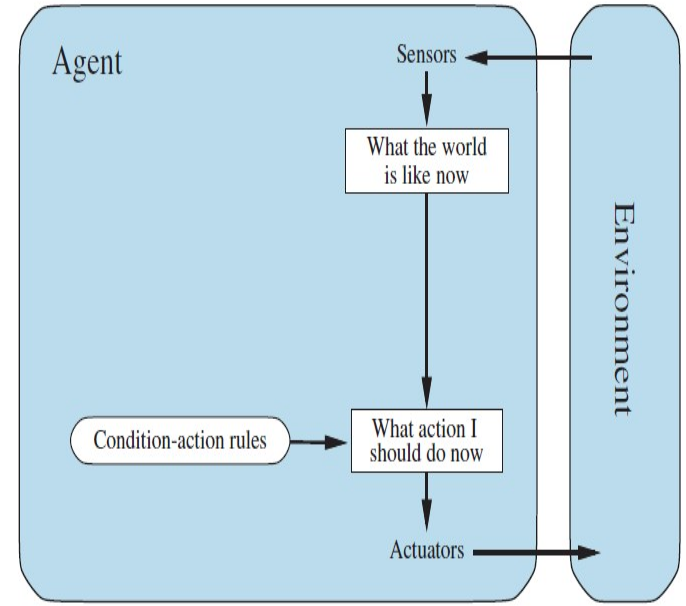
Percept (Card Inserted, PIN Correct?, Balance Check)	Action
(Yes, Yes, Sufficient)	Dispense Cash
(Yes, Yes, Insufficient)	Show Insufficient
(Yes, No, -)	Retry PIN
(No, -, -)	Wait for Card

Simple-Reflex Agent

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

Figure 2.10 A simple reflex agent. It acts according to a rule whose condition matches the current state, as defined by the percept.




```
function Reflex-Vacuum-Agent([location,status]) returns an action

if status = Dirty then return Suck

else if location = A then return Right

else if location = B then return Left
```

Limitation:

The agent will work only if a correct decision can be made based solely on the current percept that is, only if the environment is fully observable.

Simple-Reflex Agent

- **No Memory/No learning or adaptation/Use condition-action rules/Simple and Fast/Cannot handle partially observable environments**

Simple-Reflex Agent

- **No Memory/No learning or adaptation/Use condition-action rules/Simple and Fast/Cannot handle partially observable environments**

Smart Outdoor Light Controller

Condition (State)	Action
MotionDetected = True AND AmbientLightLevel = Low	TurnLightOn
MotionDetected = False AND AmbientLightLevel = Low	TurnLightOff
AmbientLightLevel = High	TurnLightOff

Simple-Reflex Agent

- **No Memory/No learning or adaptation/Use condition-action rules/Simple and Fast/Cannot handle partially observable environments**

Nighttime Driving Assistant

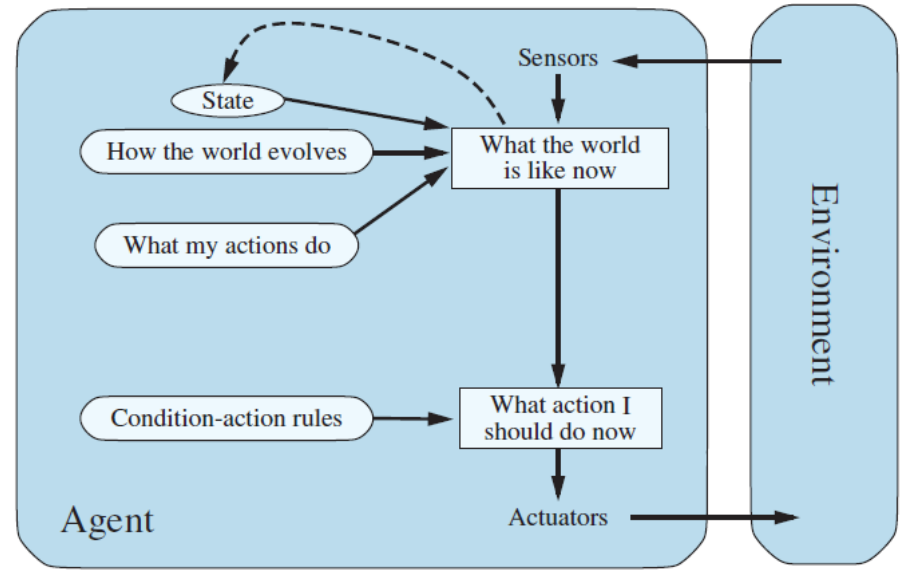
Simple reflex agents work well in controlled, predictable environments *but are less effective in complex or changing scenarios that require memory or adaptive responses.*

Model-based Reflex Agent

The most effective way to handle partial observability is for the agent to keep track of the part of the world it can't see now.
Agent maintains some Internal State

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, a description of how the next state depends on
                    the current state and action
              sensor_model, a description of how the current world state is reflected
                    in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none

  state ← UPDATE-STATE(state, action, percept, transition_model, sensor_model)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```



Limitation:

The details of how models and states are represented widely depend on the type of environment and the particular technology used in the agent design.

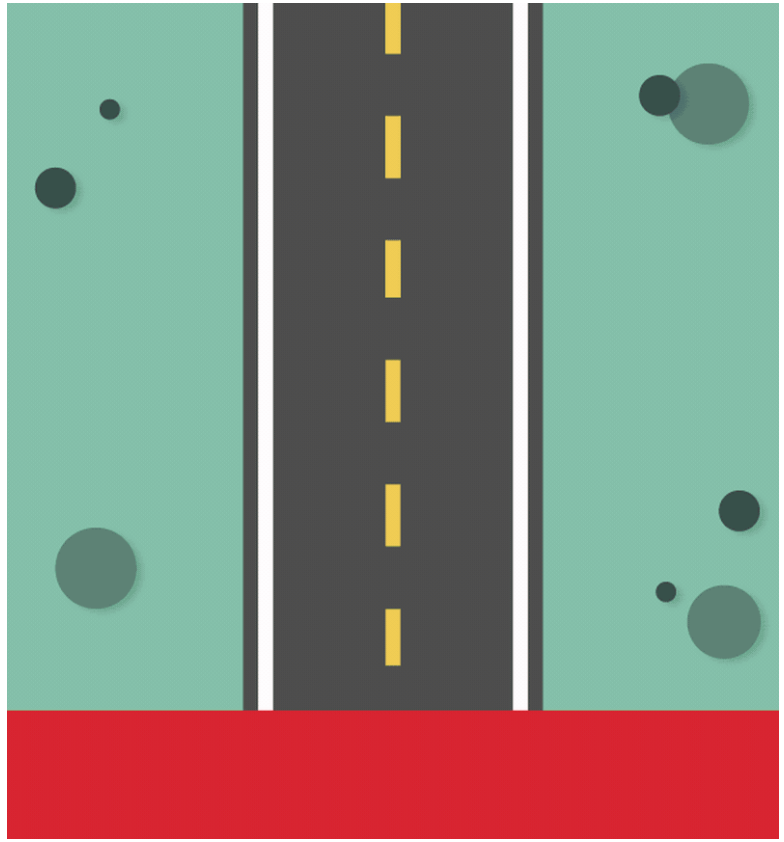
Model-based Reflex Agent

Properties:

- **Internal Model**: The agent maintains an internal model of the environment. This model represents knowledge about how the environment behaves, including how the agent's actions impact the environment.
- **State Tracking**: The agent uses its model to track the current state, even if it cannot directly observe everything.
- **Adaptable to Partially Observable Environments**: The model helps fill in gaps where direct perception is limited.
- **Limited Memory and Reasoning**: Although they use a model, these agents are generally not designed for extensive planning or future prediction. They work best in environments where immediate, short-term information is sufficient for making decisions.

Model-based Reflex Agent

- **Identify the Internal States?**



Simple Reflex Agent vs Model-based

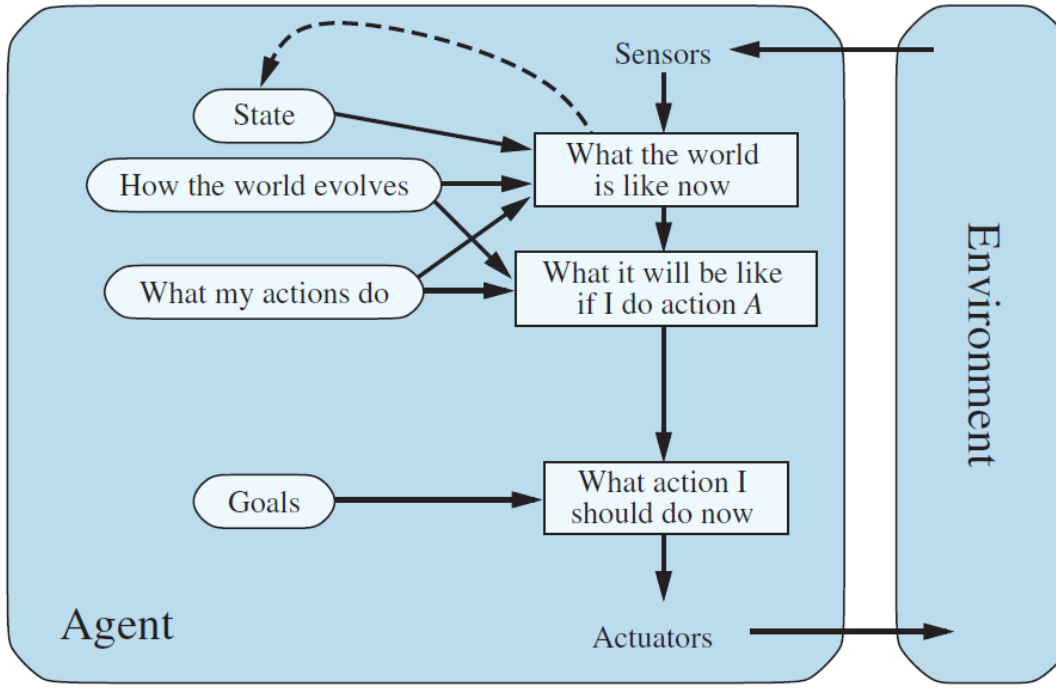
Condition (State)	Action
AmbientLightLevel = Low	TurnOnHeadlights
ObstacleDetected = True AND VehicleSpeed > 60 km/h	AlertDriver
DriverAlertnessLevel = Low	SoundAlert / Suggest Break
AmbientLightLevel = High	TurnOffHeadlights

Condition (State + Model)	Action
AmbientLightLevel = Low AND WeatherCondition = Foggy	ReduceSpeed
ObstacleDetected = True AND LanePosition = Center	MaintainLane & Brake
LanePosition = OffCenter AND NoObstacleDetected	AdjustSteering
Speed > SafeLimit AND RoadModel = Curvy	AlertDriver / SlowDown

Goal-based Reflex Agent

In addition to a description of the current state, the agent requires goal information, which outlines the situations that are considered desirable.

Reaching a goal may be a one-step action
OR
It requires searching the state space and Planning too



Goal-based Reflex Agent

Properties:

- **Goal-Oriented**: The agent has one or more goals it aims to achieve.
- **Decision-Making Based on Goals**: chooses actions that make progress toward achieving the goal, even if that means taking several intermediate steps.
- **Planning Capability**: Goal-based agents often use a planning mechanism, such as search algorithms, to determine the best sequence of actions to reach the goal.
- **Flexible and Adaptable**: These agents can adapt their behavior as conditions change in the environment.
- **Requires Computational Resources**: Goal-based agents generally require more processing power and memory than simple or model-based reflex agents, as they must assess possible outcomes and plan actions accordingly.

Goal-based Reflex Agent

- **Identify the Goal?**

Deliver the package to the GPS location X

Limitations?

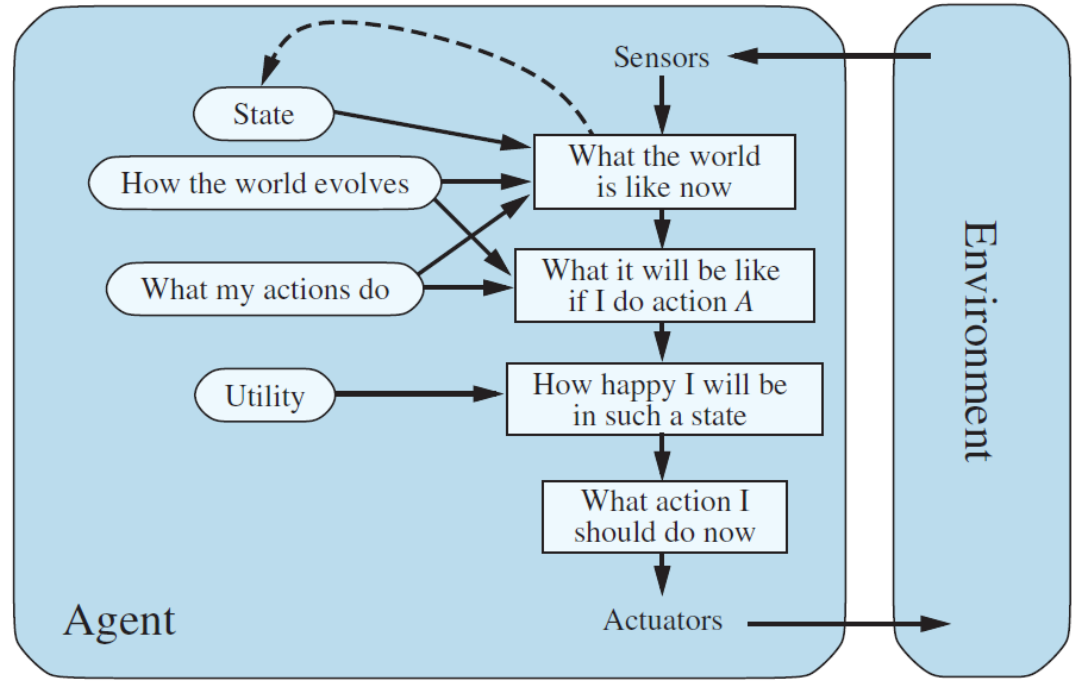
Utility-based Reflex Agent

Utility is a measurement of the happiness state of an agent.

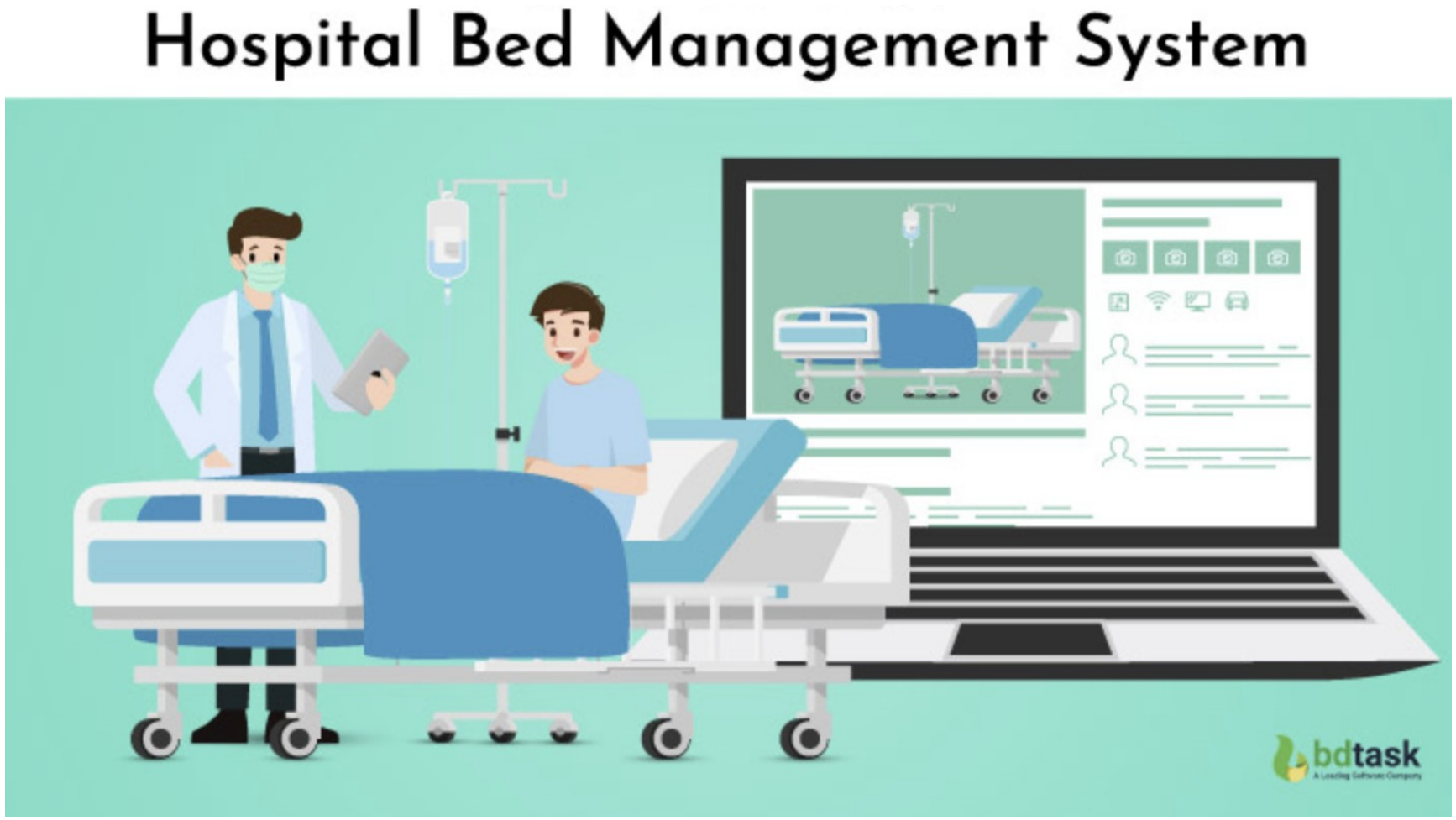
It has flexibility and learning.

Deals with conflicting goals

It weighs success over goals (sometimes)



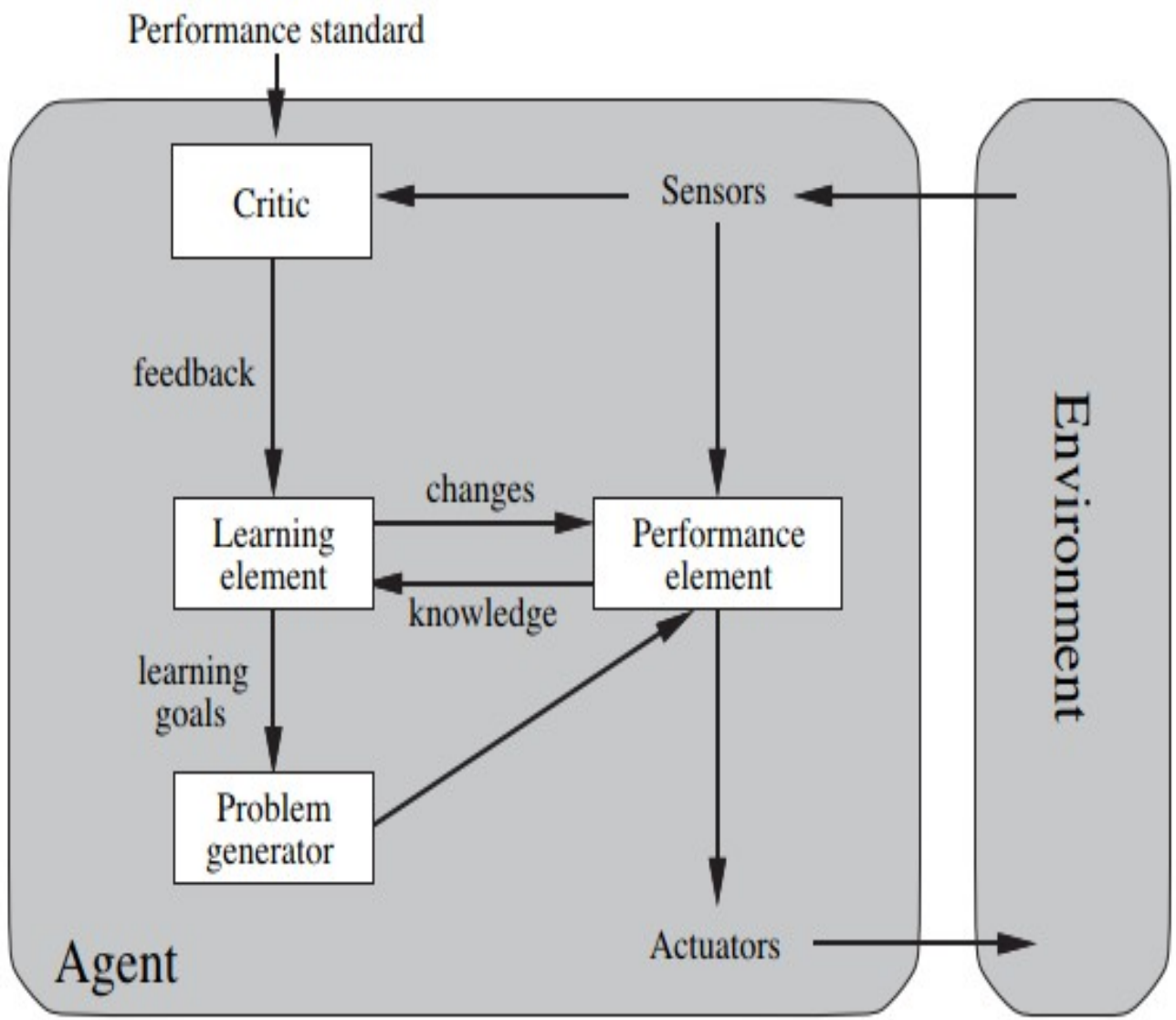
Utility-based Reflex Agent



Learning Agents

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities
- It starts to act with basic knowledge and then able to act and adapt automatically through learning
- A learning agent has mainly four conceptual components, which are:
  - Learning Element: It is responsible for making improvements by learning from environment
  - Critic: Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
  - Performance Element: It is responsible for selecting external action
  - Problem Generator: This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance

Learning Agents(Contd)



Learning Agents(Contd)

- A **learning agent** is an intelligent agent that can improve its performance over time by learning from its experiences and adapting its behavior.
- Unlike other types of agents that operate based on fixed rules or predefined models, learning agents use feedback from their actions and environment to refine their behavior, making them suitable for tasks where environments are complex, dynamic, or initially unknown.

Learning Agents(Contd)

Properties:

- **Self-Improving**: Learning agents get better over time by adapting their behavior based on past experiences. They can start with basic rules or knowledge and refine them as they gain experience in the environment.
- **Exploration and Exploitation**: Learning agents balance exploring new actions to gain more knowledge (exploration) and using known strategies to maximize rewards (exploitation). This balance is key to improving long-term performance.
- **Adaptive to Changing Environments**: Learning agents can adjust to changing conditions or rules in the environment, making them highly adaptable and versatile in dynamic situations.
- **Uncertainty Handling**: These agents can handle environments with uncertainties or incomplete information by learning from past feedback to improve future predictions and actions.

Learning Agents (Contd.)

Examples:

- **Personalized Recommendation Systems**: Over time, they adapt to changing user interests.
- **Autonomous Robots**: Robots that explore unknown environments, like Mars rovers or warehouse robots, use reinforcement learning to optimize navigation paths, avoid obstacles, and improve task completion efficiency based on experience.
- **Self-Driving Cars**: Autonomous vehicles use learning techniques to recognize traffic patterns, road signs, and obstacles, and to adapt to various driving conditions. They continuously refine their algorithms with new data collected from real-world driving.

Learning Agent (Contd.)

Examples:

- **Intelligent Virtual Assistants**: AI assistants like Siri, Alexa, and Google Assistant learn from user interactions to improve speech recognition, understand user preferences, and offer better, personalized responses.
- **Spam Detection Systems**: These systems learn to identify new spam patterns by analyzing user feedback on what constitutes spam.
- **Financial Trading Bots**: These agents learn from historical and real-time market data to make better predictions about stock prices, helping them execute trades that maximize profit or minimize risk based on their experiences.

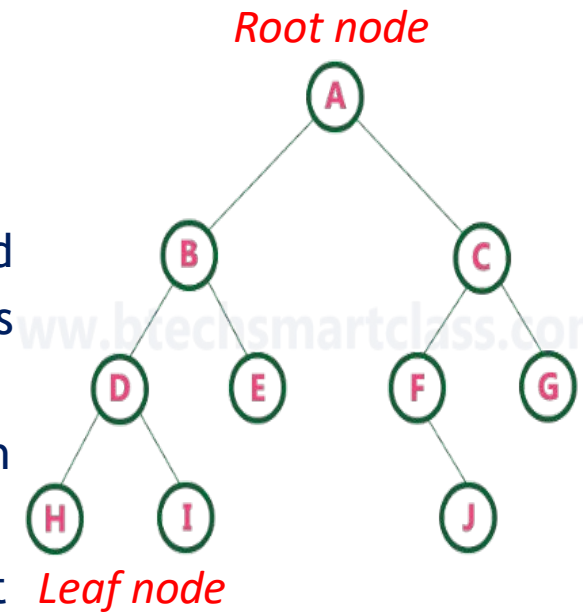
Learning Agent(Contd.)

Limitations:

- **Data Dependence**: Learning agents require a large amount of data to learn effectively. Poor or limited data can lead to inaccurate learning and poor performance.
- **Computational Demands**: Learning, especially with complex models, can be computationally expensive, requiring significant processing power and time.
- **Risk of Overfitting**: If a learning agent overfits to past experiences, it may struggle to generalize to new, unseen situations, limiting its adaptability.

Problem-Solving Agents

- In Artificial Intelligence, Search techniques are universal problem-solving methods
- ***Rational agents*** *or* ***Problem-solving agents*** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result.
- Problem-solving agents are the *Goal-based agents*
- Terminologies used in Search Algorithms
  - ***Search:*** Searching is a step by step procedure to solve a search-problem
  - ***Search Tree:*** A tree representation of search problem is called *Search tree*. The root of the search tree is the *root node* which is corresponding to the *Initial state*
  - ***Path Cost:*** It is a function which assigns a numeric cost to each path
  - ***Solution:*** It is an action sequence which leads from the start node to the goal node
  - ***Optimal Solution:*** If a solution has the lowest cost among all solution



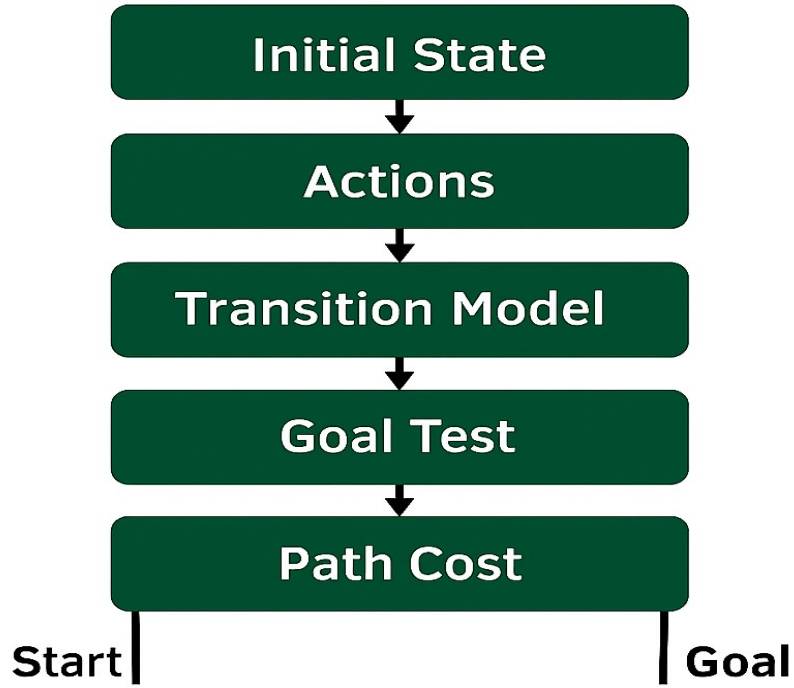
www.btechsmartclass.com

Problem-Solving Agents

Problem-Solving

Goal formulation: Organizes steps to articulate a goal.

Problem formulation: Decides actions to meet the goal.



Problem-Solving Agents

Example Problems

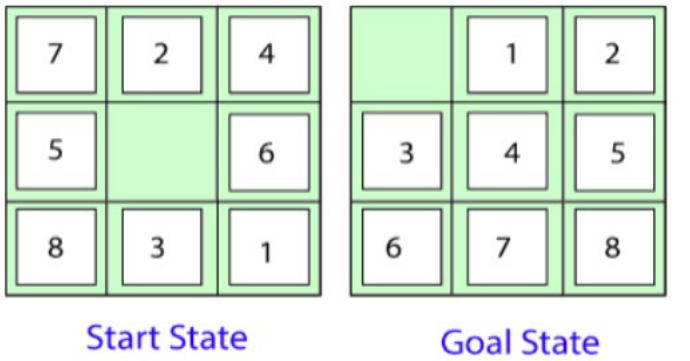
Basically, there are two types of problem approaches:

- Toy Problem: It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.
- Real-world Problem: It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

Toy Problems

8 Puzzle Problem: Here, we have a 3×3 matrix with movable tiles
numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space.

- In the figure, our task is to convert the current state into goal state by sliding digits into the blank space.

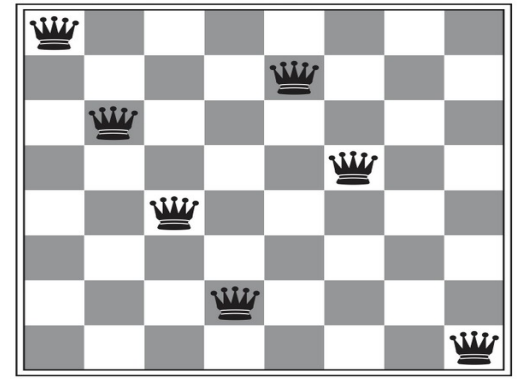


Problem-Solving Agents

- States: It describes the location of each numbered tiles and the blank tile.
- Initial State: We can start from any state as the initial state.
- Actions: Here, actions of the blank space is defined, i.e., either left, right, up or down
- Transition Model: It returns the resulting state as per the given state and actions.
- Goal test: It identifies whether we have reached the correct goal- state.
- Path cost: The path cost is the number of steps in the path where the cost of each step is 1.

Note: The 8-puzzle problem is a type of sliding-block problem which is used for testing new search algorithms in artificial intelligence.

8-queens problem



For this problem, there are two main kinds of formulation:

Incremental formulation:

1. States: Arrangement of any 0 to 8 queens on the chessboard.
2. Initial State: An empty chessboard
3. Actions: Add a queen to any empty box
4. Transition model: Returns the chessboard with the queen added in a box.
5. Goal test: Checks whether 8-queens are placed on the chessboard without any attack.
6. Path cost: There is no need for path cost because only final states are counted.

Complete-state formulation: It starts with all the 8- queens on the chessboard and moves them around, saving from the attacks.

• Following steps are involved in this formulation

1. States: Arrangement of all the 8 queens one per column with no queen attacking the other queen.
2. Actions: Move the queen at the location where it is safe from the attacks.

Search Strategies (Contd.)

→ Properties of Search Algorithms

- ***Completeness:*** A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.
- ***Optimality:*** If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution is said to be an optimal solution.
- ***Time Complexity:*** Time complexity is a measure of time for an algorithm to complete its task.
- ***Space Complexity:*** It is the maximum storage space required at any point during the search, as the complexity of the problem.

Search Strategies (Contd.)

→ **Uniformed Search**

- The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.
- It is also called *Blind search*. It examines each node of the tree until it achieves the goal node.

→ **Informed Search**

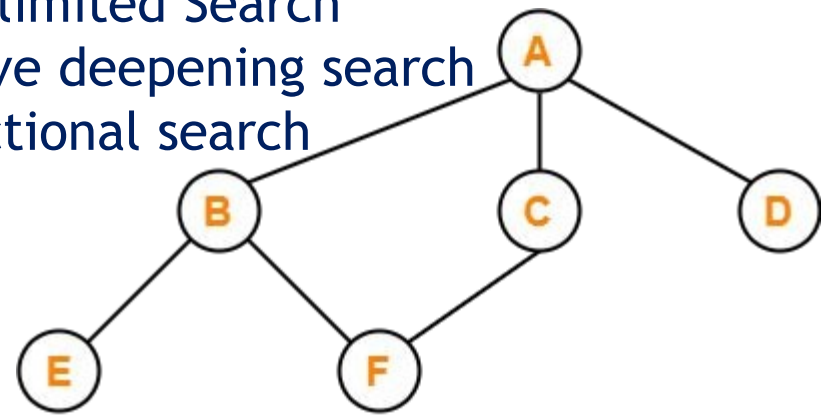
- Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search.
- Informed search strategies can find a solution more efficiently thanan uninformed search strategy.
- Informed search is also called a *Heuristic Search*

Search Strategies (Contd.)

Search Strategies

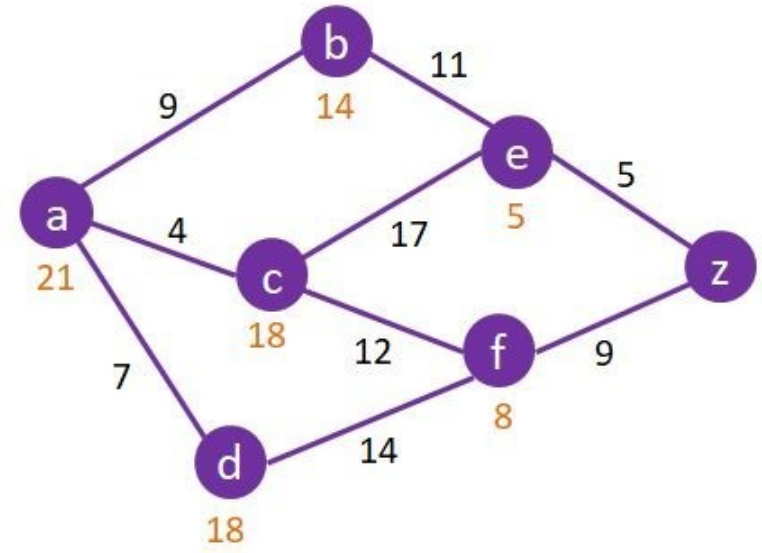
Uninformed Strategies

1. Breadth First Search
2. Uniform Cost Search
3. Depth First Search
4. Depth-limited Search
5. Iterative deepening search
6. Bidirectional search



Informed Strategies

1. Best First Search
2. A* Search

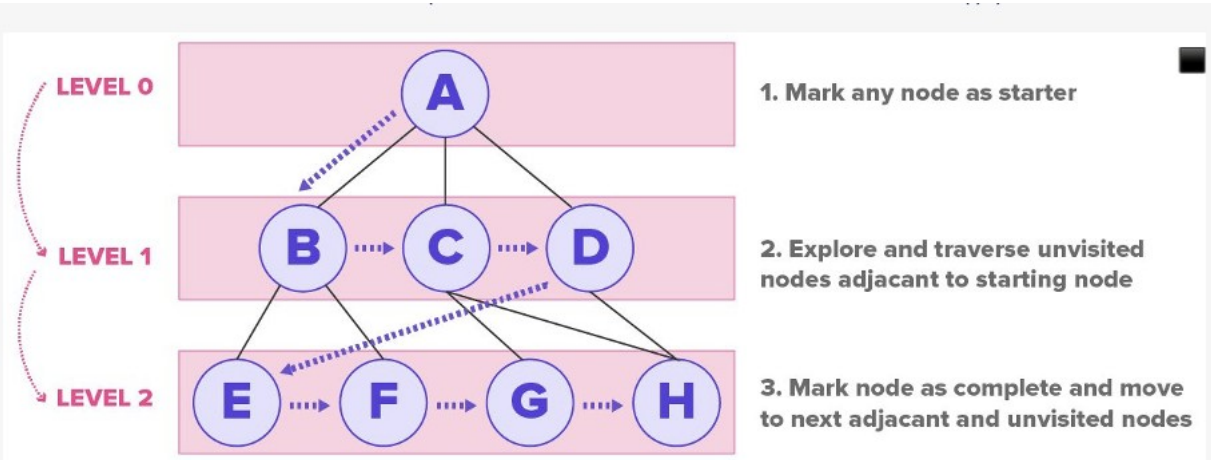


Breadth First Search (BFS)

- Breadth First Search (BFS) is the most common search strategy for traversing a tree or graph.
- This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- Breadth-first search implemented using FIFO queue data structure.
- Advantage
  - If there are more than one solutions for a given problem, then BFS will provide the minimal solution which requires the least number of steps.
- Disadvantages
  - It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
  - BFS needs lots of time if the solution is far away from the root node

BFS (Contd.)

- In BFS, root node is expanded first, then all the successors of the root node are expanded next, then their successors, and so on.
- Breadth-first search can be implemented by calling TREE-SEARCH with an empty fringe that is a first-in-first-out (FIFO) queue, assuring that the nodes that are visited first will be expanded first. In other words, calling TREE-SEARCH(*Problem*, FIFO-QUEUE()) results in a breadth-first search.



The FIFO queue puts all newly generated successors at the end of the queue, which means that shallow nodes are expanded before deeper nodes

```
function BREADTH-FIRST-SEARCH(problem) returns a solution or failure
  return GENERAL-SEARCH(problem,ENQUEUE-AT-END)
```

Depth First Search

→ Depth First Search

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure
- Depth-first search starts from the root node and follows each path to its greatest depth node before moving to the next path
- DFS uses a stack data structure for its implementation
- Advantage
  - DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
  - It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).
- Disadvantage
  - There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
  - DFS algorithm goes for deep down searching and sometime it may go to the infinite loop

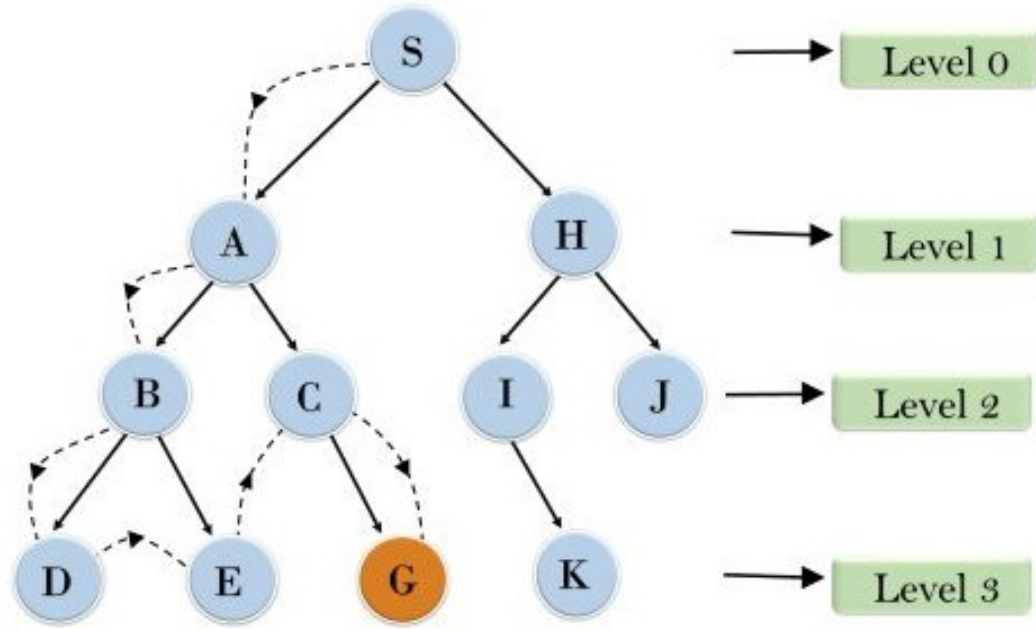
DFS(Contd.)

- It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.
- In the below search tree, we have shown the flow of depth-first search, and it will follow the order as:

Root Node → Left Node → Right Node

(or)

Root Node → Right Node → Left Node

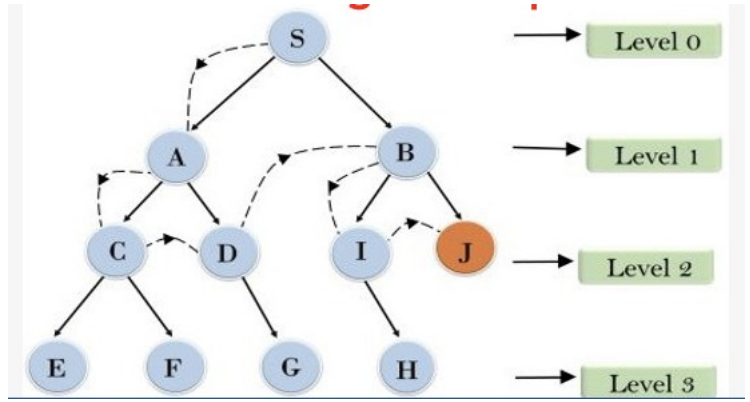


function DEPTH-FIRST-SEARCH(*problem*) **returns** a solution, or failure
GENERAL-SEARCH(*problem*,ENQUEUE-AT-FRONT)

Depth Limited Search

→ Depth-limited Search

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infini path in the Depth-first search
- In this algorithm, the node at the depth limit will treat as it has no success nodes further
- Depth-limited search can be terminated with two Conditions of failure
  - Standard failure value: It indicates that problem does not have any solution
  - Cutoff failure value: It defines no solution for the problem within a given depth limit



- Advantages
  - Depth-limited search is Memory efficient
- Disadvantages
  - It may not be optimal if the problem has more than one solution

Iterative Deepening Depth First Search

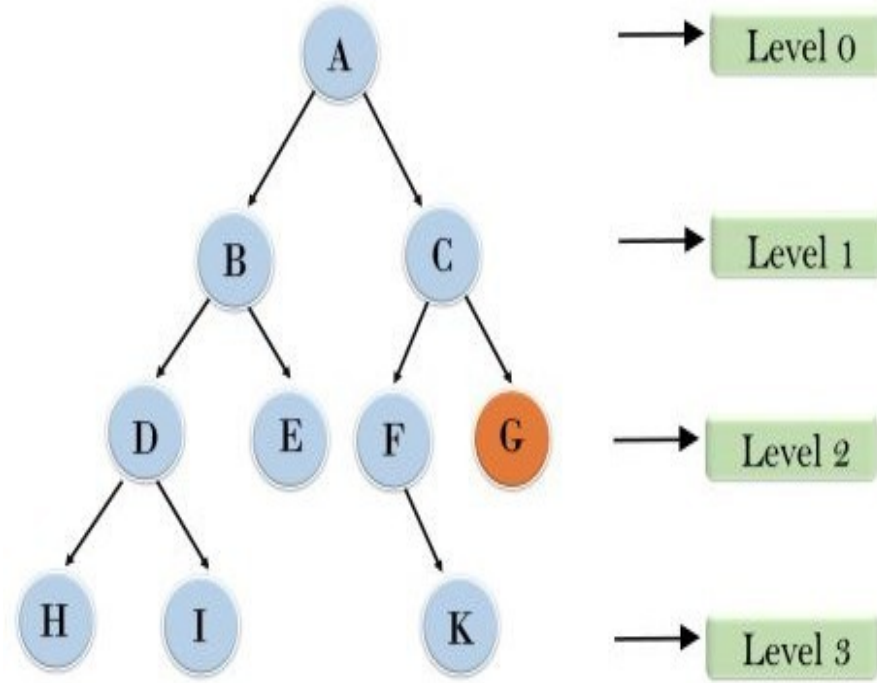
→ Iterative Deepening Depth-First Search

- The iterative deepening algorithm is a *combination of DFS and BFS* algorithms
- This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found
- This algorithm performs depth-first search up to a certain *"Depth Limit"*, and it keeps increasing the depth limit after each iteration until the goal node is found
- This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown
- Advantages
  - It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency
- Disadvantages
  - The main drawback of IDDFS is that it repeats all the work of the previous phase

Iterative Deepening DFS(Contd)

- Example
  Following tree structure is showing iterative deepening depth-first search. It performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as a

1'st Iteration → A
2'nd Iteration → A, B, C
3'rd Iteration → A, B, D, E, C, F, G
4'th Iteration → A, B, D, H, I, E, C, F, K, G




```
function ITERATIVE-DEEPENING-SEARCH( problem) returns a solution sequence
    inputs: problem, a problem

    for depth ← 0 to ∞ do
        if DEPTH-LIMITED-SEARCH( problem, depth) succeeds then return its result
    end
    return failure
```

RV College of Engineering®

Credits

1. Artificial Intelligence: A Modern Approach, Stuart J. Russell and Peter Norvig, 4th Edition
2. Artificial Intelligence basics Neeru Gupta and Ramita Mangla
3. Dr. Satish Babu, AIML, RVCE